

26 March 2026

pg 108 in JE: Greedy algorithms never work!

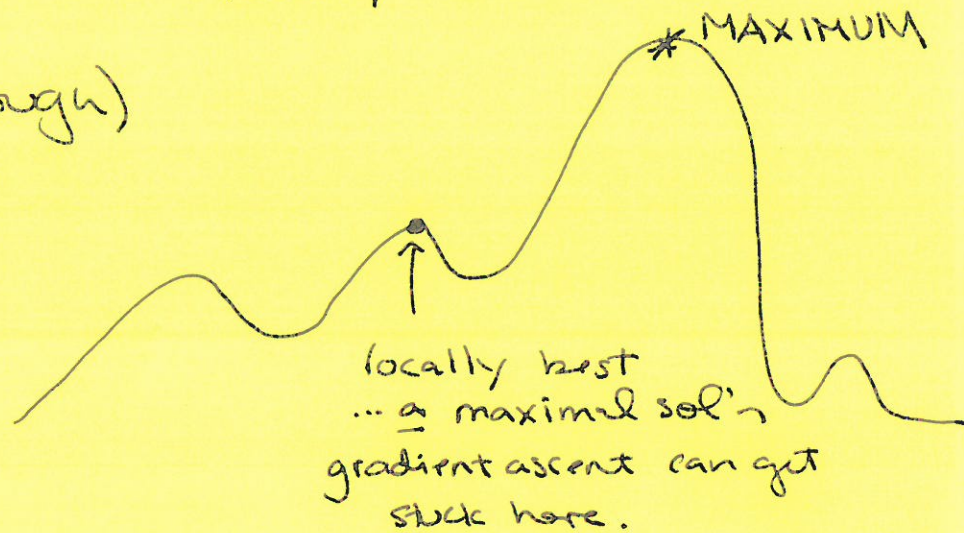
caveat: never say never...

• maximum -vs- maximal

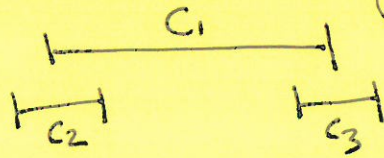
↑
THE biggest we
can get (may
not be unique though)

↑
locally best

eg, ① function



② class scheduling



• $\{c_1\}$ is a maximal class schedule: I cannot fit an additional class without overlapping times

• $\{c_2, c_3\}$ is a maximum class schedule. There is no way to take 3+ classes.

• a maximum -vs- the maximum

↑
When maximum is
not unique, we ~~need~~
can't say we've found THE one.

↑ when we know
it is unique - only
1 maximum to talk about. ①

Why does ~~it~~ finishes first work?

(general strategy: find a better sol'n than greedy and argue why greedy sol'n "stays ahead" of that other sol'n)

Proof: Suppose there are k classes in the greedy sol'n:

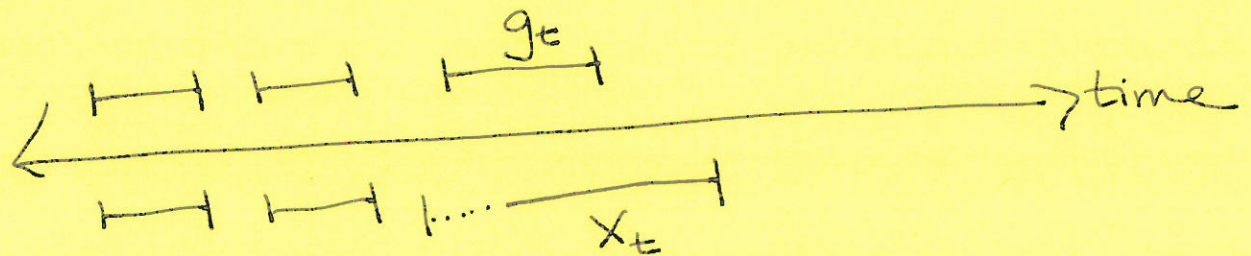
$$G = (g_1^{(s_1, f_1)}, g_2, \dots, g_k)$$

ordered by $f_1 < f_2 < f_3 < \dots < f_k$.

Let $X = (x_1, x_2, \dots, x_m)$ be a sol'n
" "
 (s_{x_1}, f_{x_1})

that is not the greedy sol'n.

Suppose t is the first index for which $g_t \neq x_t$.



If I exchange g_t for x_t in X ,

it does not conflict (Justify).

So, I can find a sol'n w/ m classes with one ~~more~~ more class from G .

Induction: either can repeat to replace all classes (2)

from G into X . Then, either $k=m$, in which case done. Or, $\exists$ class that starts after g_k , in which case we would have selected that in G . $\square$

Prefix free code:

no codeword is the prefix of another codeword
 exchange "letters", often Eng. alphabet for "codewords", oft. binary strings

Why is this helpful?

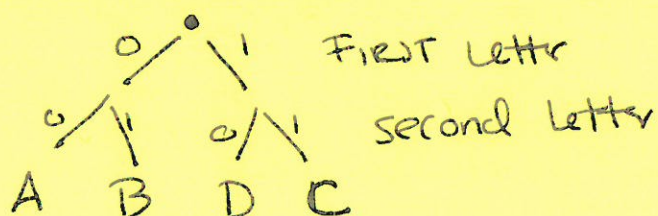
- We can use a "greedy" approach to decode, from left to right. If we see a ^{code} word (starting from left), it is the keyword.
- Nonprefix free: need to separate one letter from next.

ex: Prefix free

A	B	C	D
00	01	11	10

~~0011~~

0011 → AC
 Known how to decode



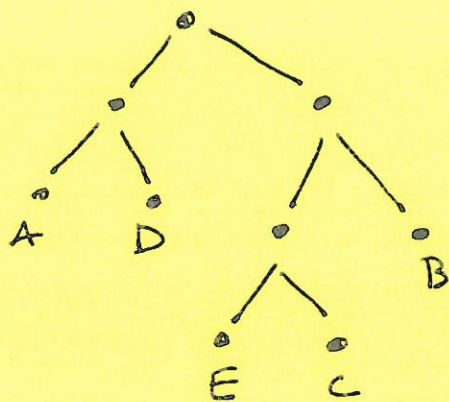
Non prefix free

A	B	C	D
0	1	01	10

0011 → ACB?
 → AABBB

unknown how to decode.

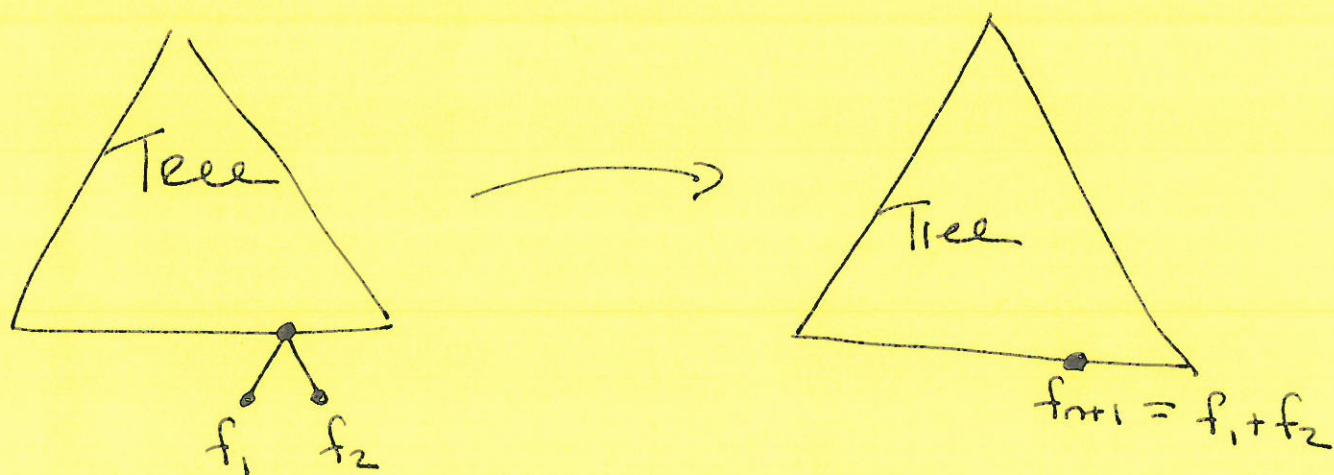
Non "linear" Huffman code:



Frequencies:

A	B	C	D	E
5	4	2	6	1

Why does this greedy strategy work?



$$f_1 < f_2 < f_3 \dots < f_n, f_{n+1}$$

INIT PROBS

New prob

$$\text{Cost} = \underbrace{\min_{\sum} \sum_{i=1}^n \text{depth}_i \cdot f_i}_{\text{in init}} = \underbrace{\min_{\sum} \sum_{i=3}^{n+1} \text{depth}_i \cdot f_i}_{\text{in "merged"}}$$

CSCI 432 Handout 11: Greedy Algorithms

Name: _____

March 26, 2026

Class Scheduling

Consider the problem of class scheduling. There are n classes available, and you want to take the maximum number of classes. Each class has a start time and a finish time:

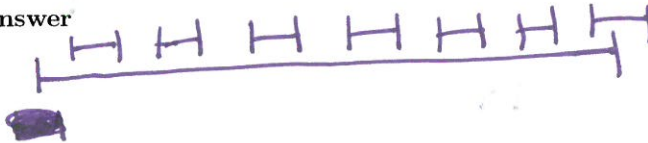
$$C = \{c_i = (s_i, f_i)\}_{i=1}^n.$$

What strategies allow you to take the most classes? For this, we will consider several different strategies. We will call a strategy optimal if it is one that maximizes the number of classes that you can take.

For the strategies listed below, are they optimal or not? If not, provide a counter-example to prove that it is not optimal.

1. Starts first: Choose the class that starts first, eliminate any conflicts with that. Among the remaining, choose the class that starts first and repeat this process until no classes are remaining.

Answer



2. Starts last.

Answer

YES

3. Finishes first.

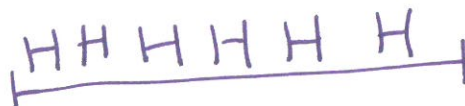
Answer

YES

4. Finishes last.

Answer

NO



5. Longest duration.

Answer

(NO) see 1 + 4.

6. Shortest duration.

Answer



7. Bonus: Randomized selection (here, find an example that shows where random selection will have a low probability of finding an optimal solution).

Answer



$$P(\text{rand. choose opt.}) = \binom{2}{52} \approx .038 < 4\%$$

4. For this code, draw a binary tree, with the encoded characters stored at the leaf nodes.

Answer

start next to table 1

5. For the Huffman strategy of "Merge the two least frequent letters and recurse", draw a the Huffman tree for the following frequency count:

Table 2: Random frequency count for letters in an imaginary text that loves weird letters.

Letter	Codeword
A	7
B	13
C	2
D	15
E	4
F	12
G	0
H	9
I	5
J	11
K	1
L	14
M	3
N	10
O	6
P	8
Q	15
R	2
S	13
T	4
U	9
V	0
W	12
X	5
Y	7
Z	11

Smaller ex:

A	B	C	D
1	5	2	3

merge + recurse

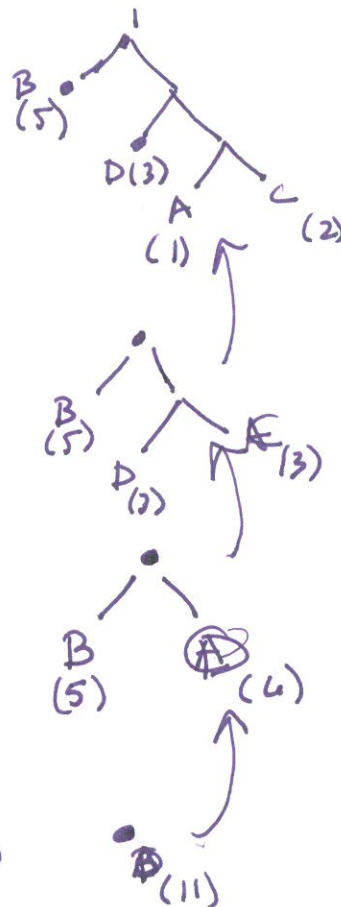
B	A	D
5	3	3

merge + recurse

B	A
5	6

merge + recurse

B
11



INPUT: alphabet + frequency (of characters in a passage)
 OUTPUT: prefix free code that uses the min # of bits for the initial passage (as given by frequency chart)
 reminds us: binary search w/ frequency problem.

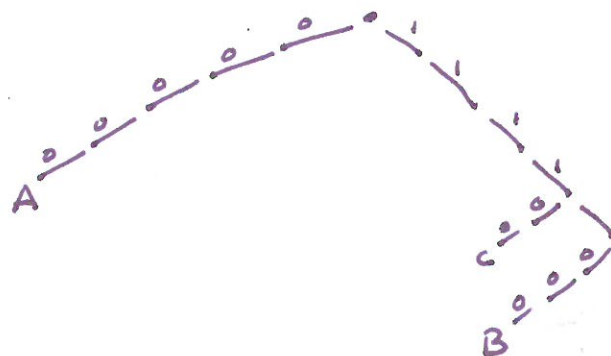
relates to union-find algo.

Hoffman Codes

Consider the following code (generated using CatChat¹):

Table 1: Prefix-free binary code for the English alphabet

Letter	Codeword
A ✓	00000
B ✓	11111000
C ✓	111100
D	1111101
E	01100
F	1111010
G	1111011
H	1100
I	0010
J	111111100
K	111111101
L	111001
M	11111001
N	0100
O	00010
P	0011
Q	0111
R	1101
S	111010
T	0101
U	111000
V	00001
W	111011
X	01101
Y	00011
Z	11111111



Prefix free code



Binary tree

3. Can you decode the following message:

01100111110010000000101110011111100101100111101000010110111111000000100100111000111010

Answer

¹Prompt was "latex table with a prefix-free code for the alphabet." The answer had a formatting problem, but was easily fixable. However, the code given was not prefix free, so I had to fix that.